



華南師範大學

本科学学生实验（实践）报告

院 系：计 算 机 学 院

实验课程：编译原理

实验项目：TINY 扩充语言的语法树生成

指导老师：黄煜廉

开课时间：2025 ~ 2026 年度第 2 学期

专 业：计算机科学与技术

班 级：24 级 计算机科学与技术 2 班

学 生：廖艺琳

学 号：20230739041

华南师范大学教务处

华南师范大学实验报告

学生姓名_____学 号_____

专 业_____年级、班级_____

课程名称_____实验项目_____

实验类型 ☐验证 ☐设计 ☐综合 实验时间_____年____月____日

实验指导老师_____实验评分_____

一、实验内容

必做内容：

（一）为 Tiny 语言扩充的语法有：

1.扩充算术表达式的运算符： $++$ （前置自增 1）、 $--$ （前置自减 1）运算符（类似于 C 语言的 $++$ 和 $--$ 运算符，但不需要完成后置的自增 1 和自减 1）、求余%、乘方 $^$ ；

2.扩充扩充比较运算符： $\leq$ （小于等于）、 $>$ （大于）、 $\geq$ （大于等于）、 $\neq$ （不等于）等运算符；

3.增加正则表达式，其支持的运算符有： $|$ （或）、 $\&$ （连接）、 $\#$ （闭包）、 $()$ （括号）、 $?$ （可选运算符）和基本正则表达式。

4.while 语句的语法规则（类似于 C 语言的 while 语言格式）：

书写格式：`while(条件) 语句序列 endwhile`

5.for 语句的语法规则（类似于 C 语言的 for 语言格式）：

书写格式：`for(循环变量赋初值;条件;循环变量自增或自减 1) 语句序列`

（二）对应的语法规则分别为：

1. $++$ （前置自增 1）、 $--$ （前置自减 1）运算符、求余%、乘方 $^$ 等运算符的文法规则请自行组织。

2. $\leq$ （小于等于）、 $>$ （大于）、 $\geq$ （大于等于）、 $\neq$ （不等于）等运算符的文法规则请自行组织。

3.为 tiny 语言增加一种新的表达式——正则表达式，其支持的运算符有：

或()
、连接(&)、闭包(#)、括号()
、可选运算符(?)和基本正则表达式,对应的文法规则请自行组织。

4.为 tiny 语言增加一种新的语句, ID::=正则表达式 (同时增加正则表达式的赋值运算符::=)

5.为 tiny 语言增加一个符合上述 while 循环的书写格式的文法规则,

6.为 tiny 语言增加一个符合上述 for 循环的书写格式的文法规则,

7.为了实现以上的扩充或改写功能,还需要注意对原 tiny 语言的文法规则做一些相应的改造处理。

二、实验目的

1. 掌握 TINY 语言文法结构、扩充文法规则设计方法,理解编译原理词法分析、语法分析核心流程。

2. 理解递归下降分析法原理,手工编写词法分析器 Lexer、语法分析器 Parser。

3. 掌握抽象语法树 AST 的节点设计、构建、遍历与图形化绘制方法。

4. 熟悉 C++ 面向对象、智能指针、容器、继承多态的工程化开发,结合 Qt 实现 GUI 界面与图形绘制。

5. 学会语法错误、词法错误的定位与提示机制,加深对编译前端错误处理的理解。

三、实验文档:

1.系统总体结构

系统采用 C++ + Qt6 开发,分为三层架构:界面交互层、词法语法核心层、AST 可视化层。

1.界面交互层(MainWindow):提供源码编辑、文件打开 / 保存、语法解析、视图切换、错误表格展示、AST 树列表与图形绘制功能,响应用户操作调用底层分析模块。

2.词法语法核心层(compiler.h/cpp):定义 Token 类型、AST 节点结构体;实现词法分析器 Lexer 完成字符扫描、分词、保留字匹配、错误捕获;实现递归下降语法分析器 Parser,完成语句与表达式递归推导、AST 构建、语法错误检测。

3.AST 可视化层：实现 AST 节点遍历、树形布局计算、自定义图形绘制，支持矩形 / 椭圆区分语句节点与表达式节点，实现兄弟节点、子节点连线布局。

2.数据结构设计

1. 词法单元（Token）与错误信息（ErrorInfo）

用于封装词法分析产出的最小语义单元，以及编译过程中产生的错误信息，是词法 / 语法分析的基础数据载体。

```
enum class TokenType {  
    // 保留字  
    IF, THEN, ELSE, END, REPEAT, UNTIL, READ, WRITE, WHILE,  
    ENDWHILE, FOR, ENDFOR, DO,  
    // 多字符标识  
    ID, NUM,  
    // 运算符/分隔符  
    ASSIGN, EQ, LT, LE, GT, GE, NE, PLUS, MINUS, TIMES, OVER,  
    MOD, POWER, LPAREN, RPAREN, SEMI, PLUSPLUS, MINUSMINUS,  
    // 正则表达式专用符号  
    REG_OR, REG_AND, REG_EMPTY, REG_QUERY, REG_ASSIGN,  
    // 辅助标识  
    ENDFILE, ERROR  
};  
  
struct Token {  
    TokenType type;    // 词法单元类型  
    string value;      // 词法单元字面量  
    int line;          // 所在行号（用于错误定位）  
};  
  
struct ErrorInfo {  
    string message;    // 错误描述
```

```

    int line;          // 错误行号
    int column;        // 错误列号
};

```

2. 抽象语法树（AST）节点

采用多态继承结构设计 AST 节点，区分语句节点（StmtNode）和表达式节点（ExpNode），适配不同语法结构的语义表达，是语法分析的核心产出物。

```

enum class NodeKind { StmtK, ExpK };          // 节点大类：语句/表达式
enum class StmtKind { IfK, RepeatK, AssignK, ReadK, WriteK, WhileK,
ForK, IncrK, RegexK }; // 语句子类
enum class ExpKind { OpK, ConstK, IdK, RegexExpK }; // 表达式子类
class ASTNode {
public:
    NodeKind nodeKind;
    int line;
    vector<shared_ptr<ASTNode>> children;
    shared_ptr<ASTNode> sibling;
    ASTNode(NodeKind kind, int ln) : nodeKind(kind), line(ln),
sibling(nullptr) {}
    virtual ~ASTNode() = default;
};
class StmtNode : public ASTNode {
public:
    StmtKind stmtKind;
    string name; // 标识符名
    StmtNode(StmtKind kind, int ln) : ASTNode(NodeKind::StmtK, ln),
stmtKind(kind) {}
};

```

```

class ExpNode : public ASTNode {
public:
    ExpKind expKind;
    TokenType op; // 运算符类型
    int val;      // 常量值
    string name;  // 标识符名
    ExpNode(ExpKind kind, int ln) : ASTNode(NodeKind::ExpK, ln),
expKind(kind) {}
};

```

3. 词法分析器（Lexer）与语法分析器（Parser）

封装词法 / 语法分析的核心逻辑，通过成员变量维护分析状态（如源码、当前位置、错误列表）。

```

class Lexer {
public:
    Lexer(const string& source);
    Token getToken(); // 获取单个词法单元
    vector<Token> tokenize(); // 全量词法分析
    vector<ErrorInfo> getErrors() const { return errors; }
private:
    string source; // 源码字符串
    size_t pos;    // 当前分析位置
    int line;      // 当前行号
    vector<ErrorInfo> errors; // 词法错误列表
    static map<string, TokenType> reservedWords; // 保留字映射表
    char peek(); // 预读当前字符
    char advance(); // 前进一个字符
    void skipWhitespace(); // 跳过空白符和注释
};

```

```

class Parser {
public:
    Parser(const vector<Token>& tokens);

    shared_ptr<ASTNode> parse();    // 语法分析入口

    vector<ErrorInfo> getErrors() const { return errors; }

private:
    vector<Token> tokens; // 词法单元列表

    size_t pos;    // 当前分析位置

    vector<ErrorInfo> errors; // 语法错误列表

    shared_ptr<ASTNode> stmt_sequence();

    shared_ptr<ASTNode> statement();

    shared_ptr<ASTNode> exp();

};

```

3.关键算法设计方案

1. 词法分析算法

采用“字符流逐字符扫描 + 状态判断”的核心逻辑，完成从源码字符串到词法单元的转换，支持保留字识别、多字符符号（如:=/:/=）、注释跳过等核心能力。

```

Token Lexer::getToken() {
    skipWhitespace();

    if (peek() == '\0') return {TokenType::ENDFILE, "", line};

    char c = advance();

    // 标识符/保留字识别
    if (isalpha(c)) {
        string s;

        s += c;

        while (isalnum(peek())) s += advance();

        if (reservedWords.count(s)) return {reservedWords[s], s, line};
    }
}

```

```

        return {TokenType::ID, s, line};
    }
    if (isdigit(c)) {
        string s;
        s += c;
        while (isdigit(peek())) s += advance();
        return {TokenType::NUM, s, line};
    }
    switch (c) {
        case '+':
            if (peek() == '+') { advance(); return {TokenType::PLUSPLUS,
"++", line}; }
            return {TokenType::PLUS, "+", line};
        case ':':
            if (peek() == '=') { advance(); return {TokenType::ASSIGN,
":=", line}; }
            if (peek() == ':') {
                advance();
                if (peek() == '=') { advance(); return
{TokenType::REG_ASSIGN, "::=", line}; }
                return {TokenType::ERROR, "::", line};
            }
            return {TokenType::ERROR, ":", line};
        // ... 其他符号匹配逻辑
        default: return {TokenType::ERROR, string(1, c), line};
    }
}

```

2. 语法分析算法（递归下降解析）

以语句序列→语句→表达式的层级递归解析为核心，适配 BNF 范式定义

的语法规则，完成词法单元到 AST 的转换。以下是 if 语句的解析实现：

```
shared_ptr<ASTNode> Parser::if_stmt() {  
    auto node = make_shared<StmtNode>(StmtKind::IfK, peek().line);  
    match(TokenType::IF); // 匹配 IF 关键字，不匹配则抛语法错误  
    node->children.push_back(exp()); // 解析条件表达式（子节点 1：条件）  
    match(TokenType::THEN); // 匹配 THEN 关键字  
    node->children.push_back(stmt_sequence()); // 解析 THEN 后的语句序列（子节点 2：then 分支）  
    if (peek().type == TokenType::ELSE) { // 可选 ELSE 分支  
        match(TokenType::ELSE);  
        node->children.push_back(stmt_sequence()); // 子节点 3：else 分支  
    }  
    match(TokenType::END); // 匹配 END 关键字  
    return node;  
}
```

3. 正则表达式语法解析

针对正则表达式的语法规则（|/&/#/?），复用递归下降框架，设计独立的解析函数（regex_exp/regex_term/regex_factor）：

```
shared_ptr<ASTNode> Parser::regex_exp() {  
    auto t = regex_term();  
    while (peek().type == TokenType::REG_OR) { // 正则或运算（最低优先级）  
        auto node = make_shared<ExpNode>(ExpKind::RegexExpK, peek().line);  
        node->op = advance().type;  
        node->children.push_back(t);  
        node->children.push_back(regex_term());  
        t = node;  
    }
```

```

    }

    return t;
}

```

4. 实验测试

{ Factorial with while loop }

read n;

fact := 1;

i := n;

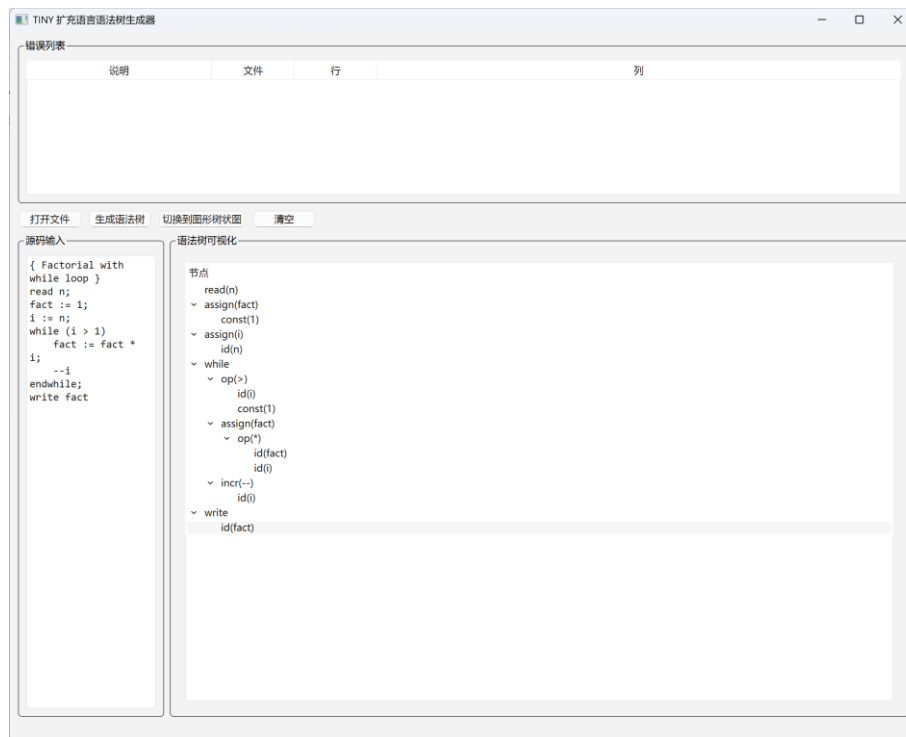
while (i > 1)

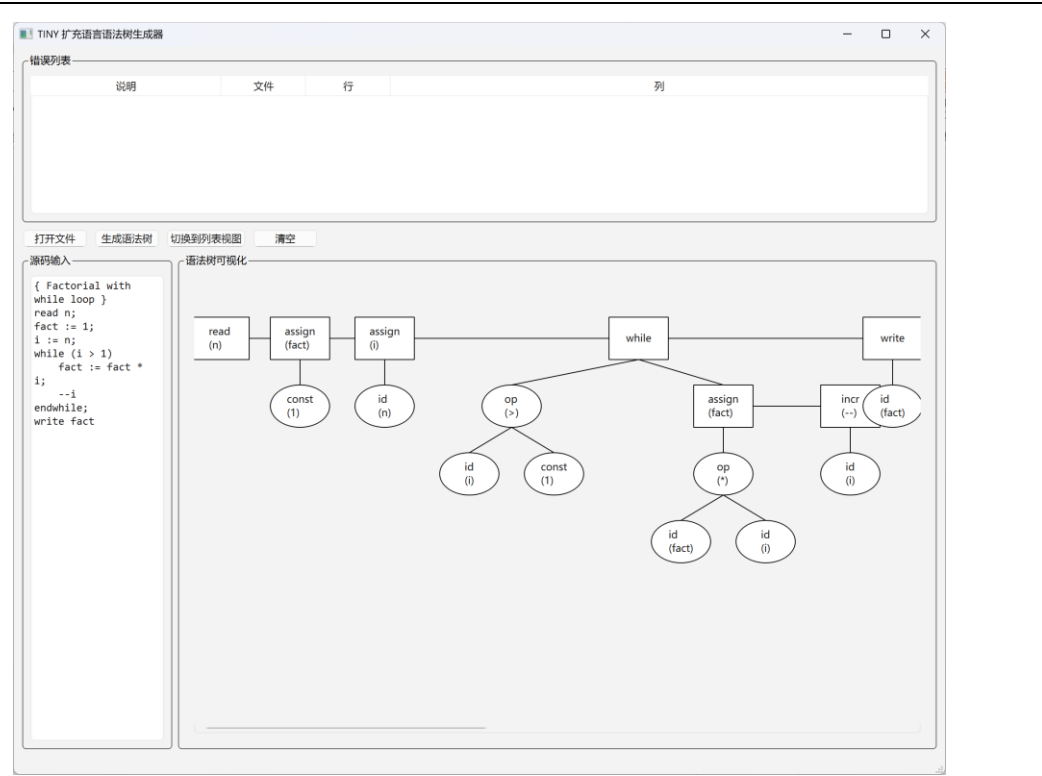
fact := fact * i;

--i

endwhile;

write fact





四、实验总结（心得体会）

1. 本次实验基于 C++ 与 Qt6 完成了 TINY 语言的语法扩充、词法语法分析、递归下降解析器编写与 AST 抽象语法树可视化开发，收获颇丰。
2. 深入掌握了编译原理前端核心流程，理解了词法分词、递归下降语法分析的底层实现逻辑，学会自主设计扩充文法 BNF 规则。
3. 熟练运用 C++ 智能指针、继承、容器与面向对象思想，完成复杂树形结构 AST 的设计、构建与遍历，提升了代码模块化设计能力。
4. 掌握 Qt GUI 界面开发、自定义图形场景绘制、树形布局算法实现，学会将算法逻辑与界面交互解耦开发。
5. 理解了词法与语法错误的检测、定位和提示机制，认识到文法设计、分支匹配严谨性对编译器正确性的重要意义。
6. 通过手动实现表达式优先级分层、循环语句与正则语句扩充，加深了对编程语言语法规则、语句嵌套、表达式结合性的理解，为后续编译原理后续学习与编译器开发打下坚实基础。

五、参考文献：

编译原理（3）——for 循环_编译原理实验三语法分析 for 循环-CSDN 博客:

<https://blog.csdn.net/zsh1184528359/article/details/109635198>

Tiny 扩充语言的语法树生成:

<https://blog.csdn.net/chaichai126/article/details/135319290>

编译原理 Tiny Syntax Tree:

<https://blog.csdn.net/phshp/article/details/139010754>